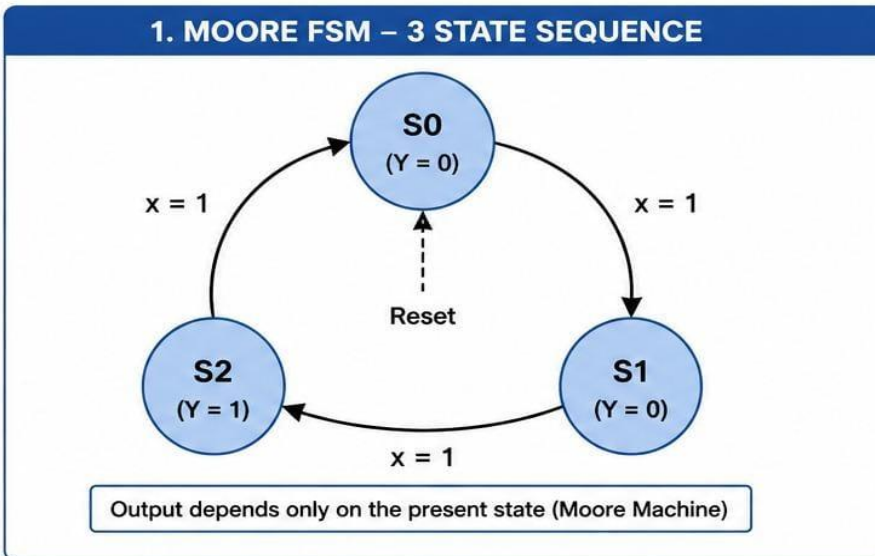


VLSI Design Internship - Task 4

RTL Design of Finite State Machines (FSM) and Control Units

Moore Finite state machine(FSM)

State Diagram:



Explanation:

The Moore FSM consists of **three states: S0, S1, and S2**. The FSM starts from **S0 when reset is activated**. At every positive edge of the clock, the FSM moves to the next state:

- S0 → S1
- S1 → S2
- S2 → S0

The starter Verilog code in the document implements these transitions using a `case(state)` statement inside a clocked `always` block with reset.

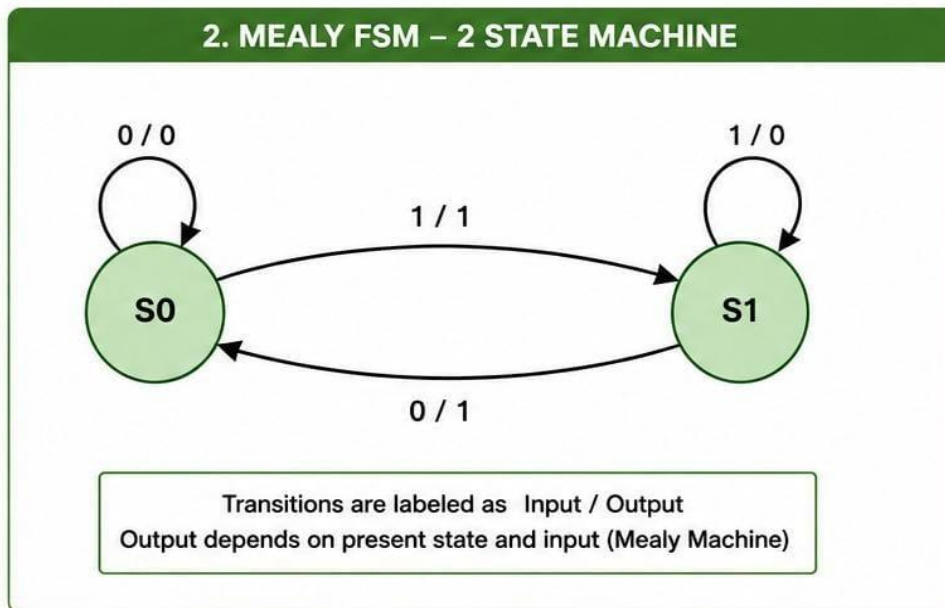
In a Moore FSM, the output is associated with the **current state**, rather than directly with the input. The document identifies Moore-vs-Mealy implementation and output logic as important FSM concepts.

Observations:

- Initially, reset places the FSM in **S0**.
- On each positive clock edge, the state changes sequentially.
- The observed sequence should be **S0 → S1 → S2 → S0**.
- The waveform should clearly show the correct state transitions.
- Reset behavior should return the FSM to S0.

Mealy Finite state machine(FSM)

State Diagram:



Explanation:

The Mealy FSM has two states, S0 and S1. According to the task document, the required condition is: Output becomes HIGH when input = 1 and current state is S1.

The FSM starts in S0 after reset.

- In S0, if input is 1, the FSM moves to S1.

- In S0, if input is 0, it remains in S0.
- In S1, if input is 1, the output becomes 1.
- In S1, if input is 0, the FSM returns to S0 and output becomes 0.

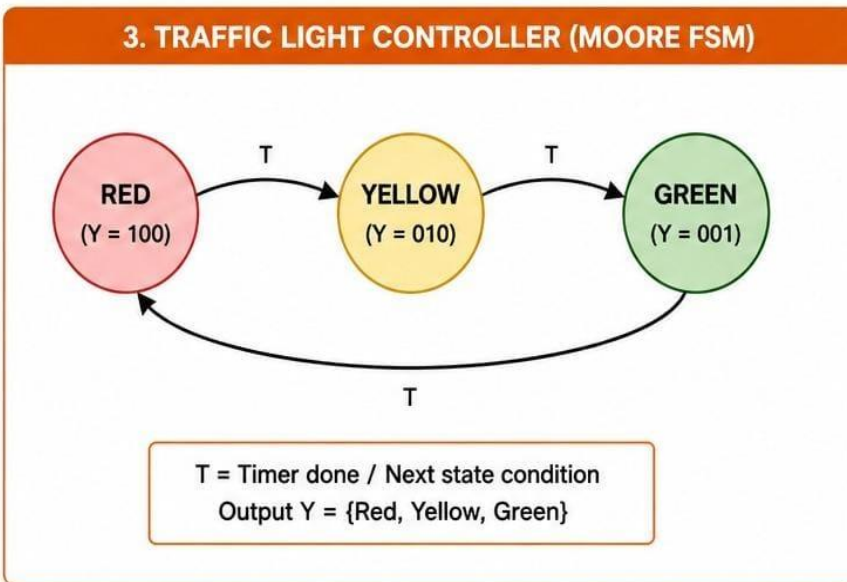
The important difference is that the Mealy output depends on both the current state and input. The document specifically asks you to understand the difference between Moore and Mealy timing and output logic.

Observations:

- Reset initially places the FSM in S0.
- Input 1 causes transition from S0 to S1.
- When the FSM is in S1 and input is 1, the output becomes HIGH.
- Input 0 from S1 returns the FSM to S0.
- The waveform should verify both state transitions and output timing.

Traffic Light Controller

State Diagram:



State Table:

State	Traffic Light
S0	RED
S1	YELLOW
S2	GREEN

The document explicitly defines the traffic-light controller states as S0 = Red, S1 = Yellow, and S2 = Green.

Explanation:

The controller contains three states:

1. **S0 – RED:** The red light is active.
2. **S1 – YELLOW:** The yellow light is active.
3. **S2 – GREEN:** The green light is active.

The controller moves sequentially through the states based on the clock/timing condition:

RED → YELLOW → GREEN → RED

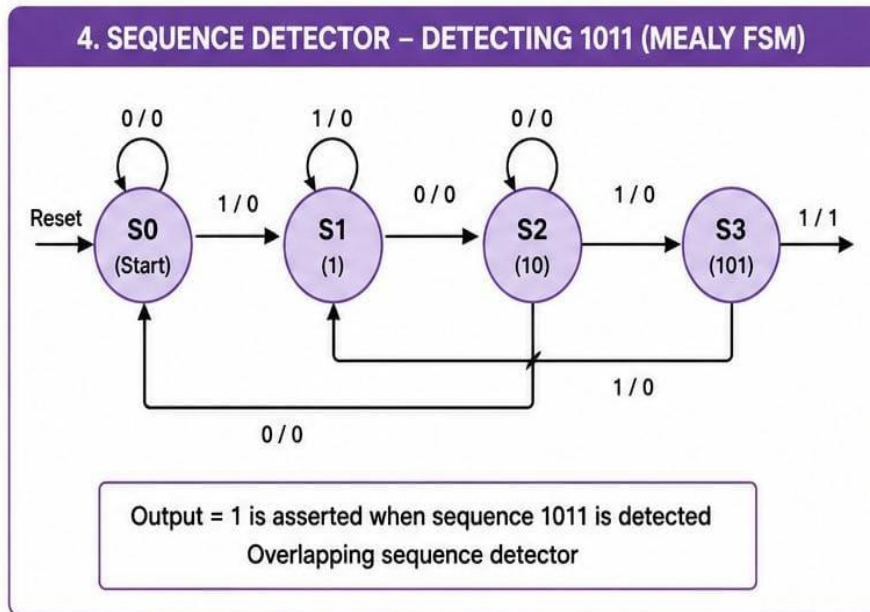
When reset is activated, the controller returns to the **RED state**.

Observations:

- After reset, the traffic light starts with **RED**.
- The controller changes from RED to YELLOW.
- It then changes from YELLOW to GREEN.
- After GREEN, it returns to RED.
- The sequence repeats continuously.
- The waveform should show the correct timing and state transitions.

Sequence Detector – 1011

State Diagram:



State Meaning:

State	Meaning
S0	No part of the sequence detected
S1	1 detected
S2	10 detected
S3	101 detected
Final 1	1011 detected

Explanation:

The objective given in the document is to **detect the binary sequence 1011 in a serial input stream.**

The FSM checks the input one bit at a time:

- **S0:** Waiting for the first 1.
- **S1:** The first 1 has been detected.
- **S2:** The sequence 10 has been detected.
- **S3:** The sequence 101 has been detected.
- When the next input is 1, the complete sequence **1011** is detected and **detected** becomes HIGH.

The document also specifically mentions studying **overlapping and non-overlapping sequence detection**.

Observations:

- Reset places the detector in **S0**.
- Input 1 moves S0 → S1.
- Input 0 moves S1 → S2.
- Input 1 moves S2 → S3.
- Input 1 completes the sequence **1011**.
- The **detected** output becomes **HIGH** when 1011 is received.
- The waveform should clearly show the input sequence, state transitions, and detection output.